Tarlac State University
COLLEGE OF COMPUTER STUDIES

Case Study in CC3 – Computer Programming 2

Submitted By:

Cabitin, Lawrence
Ignacio, Kevin
Nicdao, Archie T.
Vitug, Roel Aristotle

Submitted To:
Mr. Arvin Errol E. Liscano

Deadline March 23, 2026

Date: [March 25, 2026]

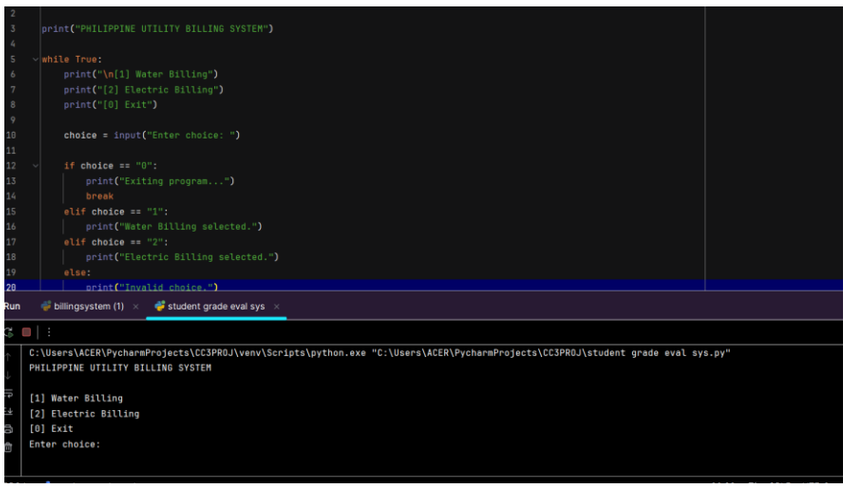
Table of Contents

A. Definition

Many households and small utility providers still experience difficulties when managing electricity and water billing manually. Traditional methods often rely on handwritten records or simple calculations, which can easily lead to problems such as incorrect bill computations, lost records, delayed billing, and a lack of transparency [1], [2]. One common issue is calculation errors. Since utility bills depend on consumptions or usages, small mistakes in inputs can result in inaccurate charges, which may cause disputes between consumers and providers [3]. Another problem is inefficient record management. Without a proper system, it becomes hard to keep track of previous bills, payment histories, and customer information, increasing the risk of misplaced data and difficulty in monitoring unpaid balances [4]. In addition, manual billing is often time-consuming. Preparing bills for multiple customers requires a lot of effort, especially when done repeatedly, which can slow down the process and lead to delays in billing and payments [2]. Lastly, the lack of organization and easy access to information makes it difficult for both providers and customers to clearly review billing details when needed [5]. To solve these issues, an electricity and water billing system is created. The system created automates bill calculations, organizes customer records efficiently, minimizes human errors, and speeds up the overall process, making billing more accurate, organized, and reliable.

B. Case Analysis (Chosen Problem)

B.1 Documentation

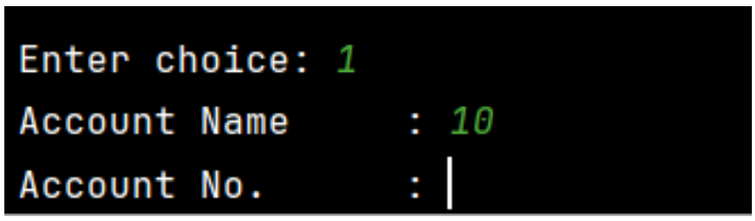


First program prototype before changing it into an array/list

```
    choice = input("Enter choice: ")

    if choice == "0":
        break

    name = input("Account Name: ")
    account_no = input("Account No: ")

    prev = float(input("Previous Reading: "))
    curr = float(input("Current Reading: "))
    consumption = curr - prev

    if choice == "1":
        total = consumption * WATER_RATE
        print("Water Bill:", total)

    elif choice == "2":
        total = consumption * ELEC_RATE
        print("Electric Bill:", total)
```

```
WATER_TIERS = [
    (10, 29.00),
    (20, 47.57),
    (30, 58.30),
    (float('inf'), 65.62),
]
WATER_SEWERAGE_RATE = 0.50
WATER_ENV_CHARGE = 2.00
WATER_VAT_RATE = 0.12

# --- ELECTRIC RATES ---
ELEC_RATE_PER_KWH = 10.95
ELEC_DISTRIBUTION = 0.175
ELEC_TRANSMISSION = 0.101
ELEC_SYSTEM_LOSS = 0.057
ELEC_TAXES = 0.117
ELEC_LIFELINE_LIMIT = 100
ELEC_LIFELINE_DISC = 0.05
```

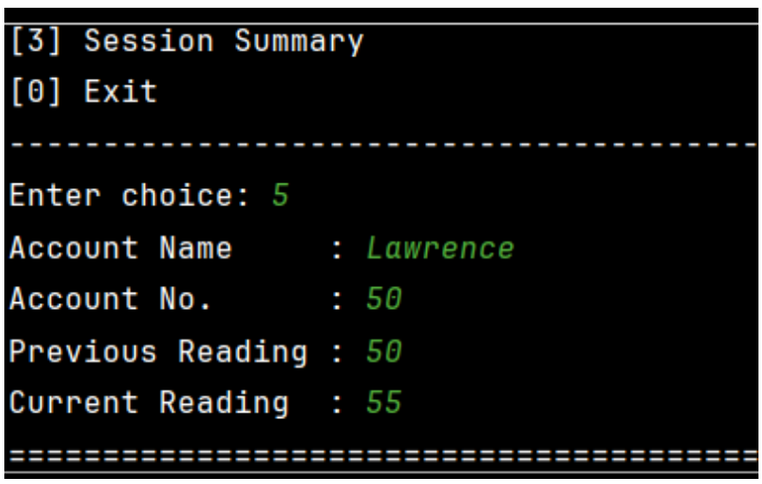
Researched Manila Water and Meralco official bill rates



First input, able to put int/floats in an input asking for a String.

```
try:
    name = input("  Account Name     : ").strip().title()
    if not name.replace( __old: " ", __new: "").isalpha():
        raise ValueError("Name must contain letters only!")
    account_no = input("  Account No.      : ").strip()
    if not account_no.isdigit():
        raise ValueError("Account number must contain digits only!")
except ValueError as e:
    print(f"  Input Error: {e}")
    continue
```

Fixed by using try and putting an exception error if the user inputs anything that isn't a string (detected by the "isalpha()" statement which returns true only if the input is in the alphabet)

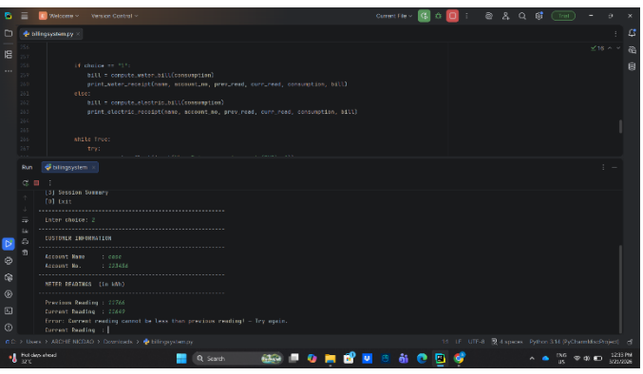


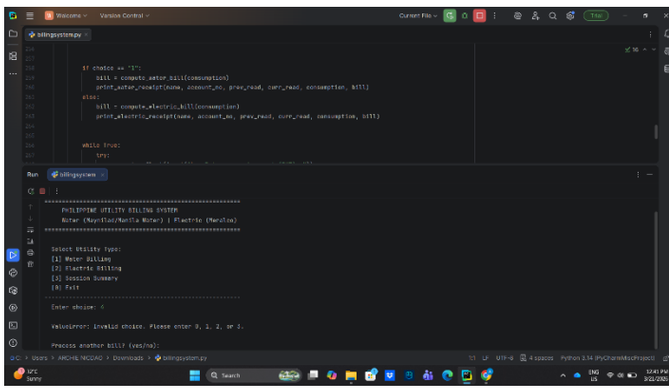
Program runs despite receiving an input not part of the possible options.

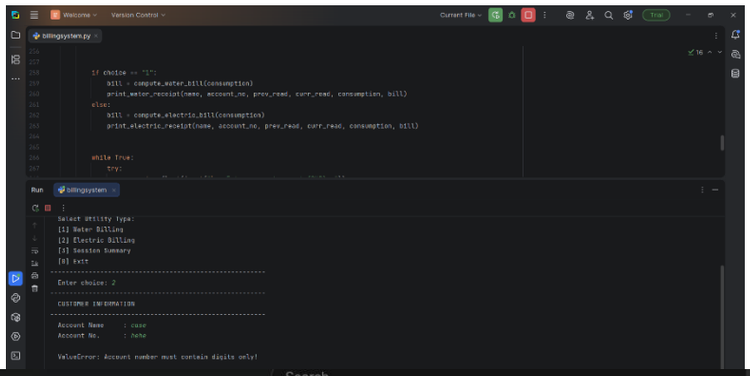
```
if choice not in ["1", "2", "3"]:
    print("  Invalid choice. Try again.")
    continue
```

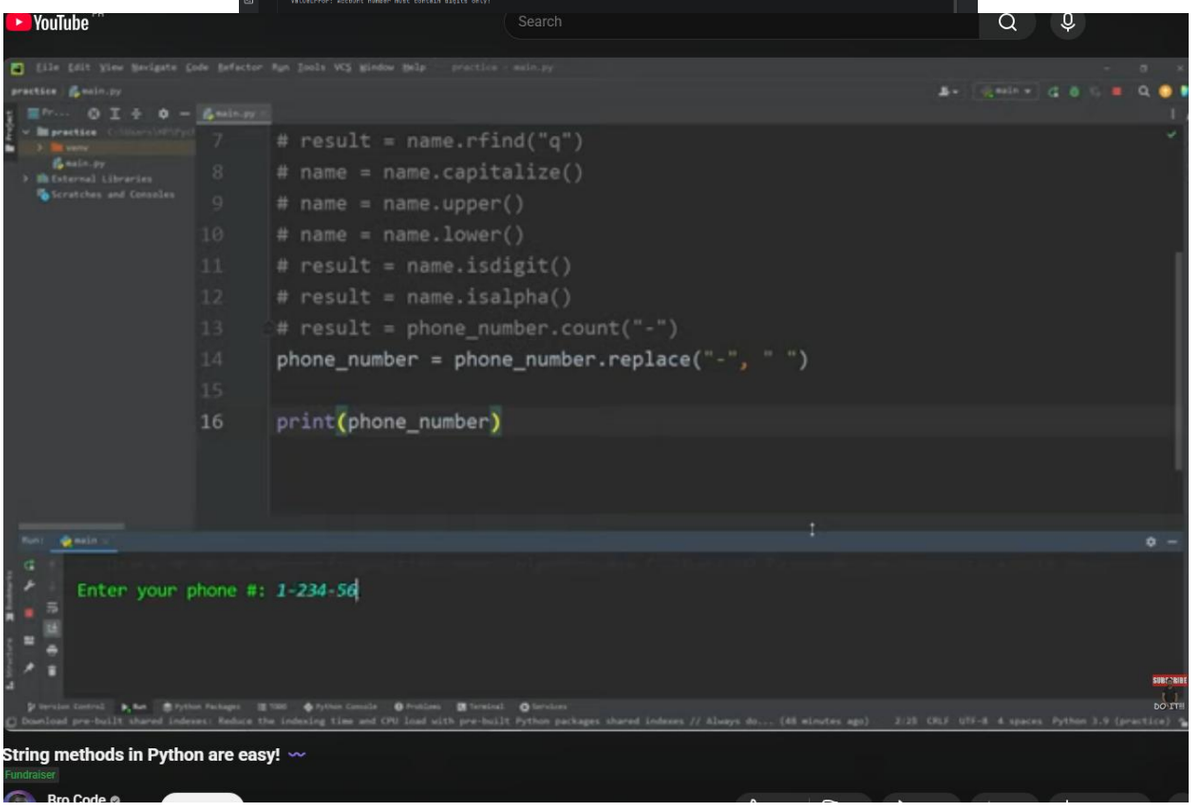
Fixed by adding list/array to ensure having only those options. And prints "Invalid choice" and iterates if this happens.

Errors fixed by try/except statements









YouTube video tutorial for some statement

B.2 Solution/Implementation

Our Utility Billing System was built with a modular, user-focused design to fix the issues caused by messy, manual record-keeping. We broke the system down into three main functional layers: The Core Engine (Persistence Layer): The entire system runs inside a perpetual while loop. We chose this so a clerk can process dozens of customers in one go without the program closing after every single bill. Finally, we added a "sentry" variable. This works as a specific exit condition, ensuring that a final summary was generated before the application was completely closed. Data Safety (Validation Layer): To deal with "dirty data," like when a user accidentally types a letter or a negative number, we wrapped our input() functions in nested try-except blocks. This acts like a safety net; if someone types a string instead of a number, the code catches the error, warns the user, and uses a continue statement to restart the prompt instead of just crashing. Billing Logic (The Calculation Layer): This is where the actual math happens. Based on our research into local utility standards, we didn't just go with simple multiplication. The system calculates a "Basic Charge," then layers on a 20% Environmental fee, and finally adds 12% VAT. This multi-step logic guarantees that the final receipt complies with Philippine official standards.

B.3 Explanation

This section describes how we created the code using the Python concepts we learned during the midterm. Data Handling (Input/Output and Type Conversion): We used the input() function to get data from the user. Then, we immediately changed the input strings into float or int types. This allowed us to do calculations with the data. For the output, we used print() combined with f-strings, utilizing \n and \t to make sure the final receipt looked organized and professional. Precise Formatting: Since we are dealing with money, accuracy is everything. We used the :.2f specifier in our strings. This forces the program to show two decimal places (like 500.00 instead of just 500), which is the standard format for any professional billing document. Decision Making (Selection): The if-elif-else structure acts as the "brain" of our app. It checks what the user wants—if they pick "Water," it runs the water tariff formulas; if they pick "Electric," it switches gears to kilowatt-hour rates. The else block is there to handle cases where a user picks an option that isn't on the menu. Loops (Iteration) - While Loop: This is a "sentinel-controlled" loop that keeps the main menu active. It only stops when the user input the exit option. For Loop: We used this for the "Session History." Every time a bill is finished, it gets saved to a list. At the very end, the for loop goes through that list and prints out a quick summary of everyone we billed that day. Error Management

(Exception Handling): We heavily used try-except blocks to manage ValueError and ZeroDivisionError. By putting our inputs inside a try block, we can catch mistakes—like someone hitting a letter key by accident—and show a friendly error message. This prevents the messy "Traceback" text that usually pops up when a Python script fails.

C. References

[1] International Energy Agency, Digitalization and Energy Efficiency, 2022.

[2] World Bank, Improving Utility Services Through Digital Systems, 2021.

[3] Institute of Electrical and Electronics Engineers, “Automated Billing Systems in Utility Management,” 2020.

[4] Management Information Systems by Kenneth C. Laudon and Jane P. Laudon, 15th ed. Pearson, 2018.

[5] Philippine Statistics Authority, Household Utilities and Consumption Data, 2023

Group Contribution:

Cabitin – Program

Vitug - Definitions, references, Bug tester

Ignacio - Solutions, explanation, Bug tester

Nicdao – Documentation, Bug tester